Lecture 10 — Number Theory I: Divisibility, gcd/lcm, Primes, Sieve, Factorization

> Year Curriculum · Prerequisites: Week 3 (complexity), Week 9 (bit manipulation is helpful but not required) · Estimated time: 2 × 90 min

1. Overview

Number theory problems appear in almost every contest, from a Div. 2 A ("is n a product of two primes?") to the hardest problem in a set. The good news is that a small toolkit covers the vast majority of them:

- divisor enumeration and primality testing in O(√n);
- the Euclidean algorithm for gcd, and lcm without overflow;
- the Sieve of Eratosthenes and its variant, the smallest-prime-factor (SPF) sieve;
- prime factorization, and the formulas that turn a factorization into the number and sum of divisors;
- the "harmonic sieve" pattern: for each d, loop over its multiples, which costs only O(M log M).

Next week (Week 11) builds modular arithmetic on top of this: fast exponentiation, modular inverses and binomial coefficients modulo a prime. The Regionals track later adds the extended Euclidean algorithm, CRT and Euler's totient.

2. Divisibility and the fundamental theorem

We write a | b ("a divides b") if b = a·k for some integer k. Divisibility is preserved by linear combinations: if a | b and a | c then a | bx + cy for any integers x, y. This single fact powers Euclid's algorithm.

A **prime** is an integer p > 1 whose only positive divisors are 1 and p. The **fundamental theorem of arithmetic** says every integer n > 1 has a unique factorization

$n = p_1^{e_1} \cdot p_2^{e_2} \cdot \ldots \cdot p_k^{e_k}$, with $p_1 < p_2 < \ldots < p_k$ prime and $e_i \geq 1$.

For example $360 = 2^3 \cdot 3^2 \cdot 5$. A divisor of n is obtained by choosing, independently for each prime, an exponent f_i with $0 \leq f_i \leq e_i$. Almost every divisor-related formula follows from this picture.

Useful size facts for estimating complexity:

- π(n), the number of primes up to n, is about n / ln n: $\pi(10^6) = 78{,}498$ and $\pi(10^7) = 664{,}579$.
- A number $n \le 10^9$ has at most 9 distinct prime factors (the product of the first ten primes, 6,469,693,230, exceeds 10^9), and at most 1,344 divisors. For $n \le 10^{12}$ the maximum is 6,720 divisors. So "loop over all divisors" is cheap once you have them.
- n has at most one prime factor larger than √n.

3. O(√n) algorithms

3.1 Divisor enumeration

If d divides n, so does n/d, and $\min(d, n/d) \le \sqrt{n}$. So it suffices to try d = 1, 2, … while $d^2 \le n$, and record both d and n/d:

```
vector<long long> divisors(long long n) {
    vector<long long> small, large;
    for (long long d = 1; d * d <= n; d++)
        if (n % d == 0) {
            small.push_back(d);
            if (d != n / d) large.push_back(n / d);
        }
    small.insert(small.end(), large.rbegin(), large.rend());   // sorted order
    return small;
}
```

Two details matter. First, write the loop condition as `d * d <= n` in integer arithmetic, not `d <= sqrt(n)`: `sqrt` returns a double, and for large perfect squares rounding can make you miss or overshoot the last value. Second, when n is a perfect square, d = √n pairs with itself; the `d != n / d` check prevents listing it twice. For $n = 10^{12}$ the loop runs 10^6 times.

3.2 Primality testing

The same bound gives a primality test: $n \ge 2$ is prime iff no d in [2, √n] divides it. That is fast enough for a handful of numbers up to about 10^{14}. For many queries, use a sieve (section 5). For a single n up to 10^{18}, $\sqrt{n} = 10^9$ is too slow, and the right tool is the Miller–Rabin test, which appears later in the course.

3.3 Factorization by trial division

```
vector<pair<long long, int>> factorize(long long n) {
    vector<pair<long long, int>> f;
    for (long long p = 2; p * p <= n; p++) {
        if (n % p) continue;
        int e = 0;
        while (n % p == 0) { n /= p; e++; }
        f.push_back({p, e});
```

```
    }
    if (n > 1) f.push_back({n, 1});
    return f;
}
```

Why does this only ever record primes, even though p runs over all integers? When the loop reaches a composite p, all of p's prime factors are smaller than p and were already divided out of n, so p cannot divide the current n. Why the final `if (n > 1)`? After the loop, the remaining n has no prime factor ≤ √(remaining n), so it is either 1 or a single prime — the one possible "big" prime factor. Note the loop condition uses the *current* n, which shrinks as factors are removed, so the loop often ends early.

Example: 600851475143 = 71 · 839 · 1471 · 6857. The loop stops shortly after p = 1471, when p^2 exceeds the remaining value 6857, instead of running to √600851475143 ≈ 775,146.

4. gcd and lcm

4.1 Euclid's algorithm

The greatest common divisor satisfies gcd(a, b) = gcd(b, a mod b) and gcd(a, 0) = a. Proof sketch: a mod b = a − qb is a linear combination of a and b, so every common divisor of a and b divides a mod b; conversely a = qb + (a mod b), so every common divisor of b and a mod b divides a. The two pairs have exactly the same common divisors.

```
gcd(84, 36) = gcd(36, 12) = gcd(12, 0) = 12
```

```
long long gcdRec(long long a, long long b) { return b == 0 ? a : gcdRec(b, a % b); }
```

In C++17 just use `std::gcd` from `<numeric>` (included by `bits/stdc++.h`). Complexity: when a ≥ b, a mod b < a/2, so the larger argument at least halves every two steps, giving **O(log min(a, b))**. The worst case is a pair of consecutive Fibonacci numbers.

4.2 lcm and identities

- gcd(a, b) · lcm(a, b) = a · b, so lcm(a, b) = **a / gcd(a, b) · b**. Dividing first keeps the intermediate value at most the final answer; `a * b / gcd(a, b)` can overflow even when the lcm fits.
- In terms of factorizations, gcd takes the minimum exponent of each prime and lcm the maximum.
- gcd is associative: the gcd of an array is a left fold, `g = gcd(g, a[i])`. It only decreases, and each strict decrease at least halves it, so a running gcd changes at most O(log) times — a fact used in many "gcd of subarrays" problems.

- gcd(a, b) = gcd(a, b − a). Hence gcd($a_1 + x$, $a_2 + x$, …, $a_n + x$) = gcd($a_1 + x$, $a_2 - a_1$, …, $a_n - a_1$): precompute the gcd of the differences once and answer each x with one more gcd. This is Codeforces 1458A *Row GCD*.

5. Sieves

5.1 Sieve of Eratosthenes

To find all primes up to n, start with every number marked prime, and for each prime p cross out its multiples. Two optimisations make it fast: the outer loop only needs to go to √n, and the inner loop can start at p^2, because any multiple kp with k < p has a prime factor smaller than p and was crossed out already.

```
vector<bool> isPrime(n + 1, true);
isPrime[0] = false; if (n >= 1) isPrime[1] = false;
for (int i = 2; (long long)i * i <= n; i++)
    if (isPrime[i])
        for (int j = i * i; j <= n; j += i) isPrime[j] = false;
```

The work is $\Sigma_{p \le n}\ n/p = n \cdot \Sigma\ 1/p \approx n \ln \ln n$, which is practically linear: about $3 \cdot 10^7$ operations for $n = 10^7$. `vector<bool>` stores one bit per entry, so $n = 10^8$ needs only 12.5 MB.

5.2 Smallest prime factor sieve

If, instead of a boolean, we store for every x the smallest prime that divides it, we can factorize any x ≤ n in O(log x): divide by `spf[x]` repeatedly. Each division at least halves x.

```
spf: 84 → 2,  42 → 2,  21 → 3,  7 → 7
84 = 2² · 3 · 7
```

This is the method of choice when you must factorize many numbers up to ~10^7: 10^5 numbers cost about $2 \cdot 10^6$ divisions after a one-time sieve. Memory is the constraint: an `int` array of size 10^7 is 40 MB.

The **linear sieve** computes the same SPF array in O(n) by crossing out each composite exactly once (from its smallest prime factor), and produces the list of primes as a by-product. It is shown on the slides; in practice it runs at a similar speed to Eratosthenes, so use whichever you remember correctly.

5.3 Harmonic sieves

Many tasks need some divisor-related quantity for every x ≤ M. Loop over every d and over its multiples:

```
for (int d = 1; d <= M; d++)
    for (int j = d; j <= M; j += d)
        cnt[j]++;          // d is a divisor of j
```

The inner loop runs M/d times, so the total is M(1 + 1/2 + … + 1/M) = M · H_M ≈ M ln M ≈ 1.4 · 10^7 for M = 10^6. Replacing `cnt[j]++` by `sum[j] += d` computes the sum of divisors of every number.

The pattern also works in reverse, "for each candidate d, look at all multiples of d present in the input". **Worked example — CSES 1081 Common Divisors:** given up to 2 · 10^5 numbers ≤ 10^6, find the largest gcd of any pair. The gcd of a pair equals g only if both numbers are multiples of g; conversely, if at least two input numbers are multiples of g, their gcd is at least g. So the answer is the largest g with at least two multiples in the input. Count occurrences in `cnt[]`, then for g from 10^6 down to 1 add up `cnt[g] + cnt[2g] + …`; the first g reaching 2 is the answer. Total O(M log M). For input `3 14 15 7 9`, g = 7 is the first value with two multiples (14 and 7).

5.4 Segmented sieve

To list primes in [L, R] where R is up to 10^{12} but R − L is at most ~10^6, sieve the primes up to √R first, then use each of them to cross out its multiples inside a boolean array representing [L, R]. The first multiple of p to cross out is max(p^2, ⌈L/p⌉ · p). Do not forget that 1 is not prime if L = 1. Total work is O((R − L) log log R + √R). SPOJ PRIME1 is the classic exercise.

6. From a factorization to divisor functions

With n = Π p_i^e_i:

- **Number of divisors**: d(n) = Π (e_i + 1), since each exponent is chosen independently from e_i + 1 options. d(360) = 4 · 3 · 2 = 24.
- **Sum of divisors**: σ(n) = Π (1 + p_i + p_i^2 + … + p_i^e_i) = Π (p_i^(e_i+1) − 1)/(p_i − 1). σ(360) = 15 · 13 · 6 = 1170.
- **Product of divisors**: n^(d(n)/2) (pair each divisor with its complement). Computing it modulo a prime needs Week 11 (CSES 2182 *Divisor Analysis*).
- **Perfect squares** are exactly the numbers with all exponents even, which is also exactly the numbers with an odd number of divisors. Codeforces 230B *T-primes* uses this: x has exactly three divisors iff x = p^2 for a prime p.

7. Reference implementations

7.1 Toolkit: sieve, SPF, trial-division factorization, gcd/lcm

```cpp
#include <bits/stdc++.h>
using namespace std;
using ll = long long;

// Sieve of Eratosthenes: isPrime[i] for 0..n
vector<bool> sieve(int n) {
    vector<bool> isPrime(n + 1, true);
    isPrime[0] = false;
    if (n >= 1) isPrime[1] = false;
    for (int i = 2; (ll)i * i <= n; i++)
        if (isPrime[i])
            for (int j = i * i; j <= n; j += i) isPrime[j] = false;
    return isPrime;
}

// Smallest prime factor sieve: spf[x] = smallest prime dividing x (x >= 2)
vector<int> spfSieve(int n) {
    vector<int> spf(n + 1, 0);
    for (int i = 2; i <= n; i++)
        if (spf[i] == 0)
            for (int j = i; j <= n; j += i)
                if (spf[j] == 0) spf[j] = i;
    return spf;
}

// Trial division: O(sqrt(n)). Returns (prime, exponent) pairs.
vector<pair<ll, int>> factorize(ll n) {
    vector<pair<ll, int>> f;
    for (ll p = 2; p * p <= n; p++) {
        if (n % p) continue;
        int e = 0;
        while (n % p == 0) { n /= p; e++; }
        f.push_back({p, e});
    }
    if (n > 1) f.push_back({n, 1});
    return f;
}

// Factorize with SPF table: O(log x)
vector<pair<int, int>> factorizeSpf(int x, const vector<int>& spf) {
    vector<pair<int, int>> f;
    while (x > 1) {
        int p = spf[x], e = 0;
        while (x % p == 0) { x /= p; e++; }
        f.push_back({p, e});
    }
    return f;
}
```

```cpp
ll lcmll(ll a, ll b) { return a / gcd(a, b) * b; }   // divide first to avoid overflow

int main() {
    auto isPrime = sieve(50);
    cout << "primes <= 50:";
    for (int i = 0; i <= 50; i++) if (isPrime[i]) cout << ' ' << i;
    cout << "\n";

    for (ll x : {360LL, 97LL, 1000000007LL * 2, 600851475143LL}) {
        cout << x << " =";
        for (auto [p, e] : factorize(x)) cout << ' ' << p << '^' << e;
        cout << "\n";
    }

    auto spf = spfSieve(100);
    cout << "84 via spf =";
    for (auto [p, e] : factorizeSpf(84, spf)) cout << ' ' << p << '^' << e;
    cout << "\n";

    cout << "gcd(84, 36) = " << gcd(84LL, 36LL) << ", lcm = " << lcmll(84, 36) << "\n";
}
```

Output:

```
primes <= 50: 2 3 5 7 11 13 17 19 23 29 31 37 41 43 47
360 = 2^3 3^2 5^1
97 = 97^1
2000000014 = 2^1 1000000007^1
600851475143 = 71^1 839^1 1471^1 6857^1
84 via spf = 2^2 3^1 7^1
gcd(84, 36) = 12, lcm = 252
```

7.2 CSES 1713 — Counting Divisors (harmonic sieve)

```cpp
#include <bits/stdc++.h>
using namespace std;
// CSES 1713 - Counting Divisors: n queries, each x <= 1e6, print d(x).
// Precompute d(x) for all x with a harmonic-series sieve: O(M log M).
int main() {
    ios::sync_with_stdio(false); cin.tie(nullptr);
    const int M = 1000000;
    vector<int> d(M + 1, 0);
    for (int i = 1; i <= M; i++)
        for (int j = i; j <= M; j += i) d[j]++;
    int n; cin >> n;
    while (n--) {
        int x; cin >> x;
        cout << d[x] << '\n';
    }
}
```

7.3 CSES 1081 — Common Divisors

```cpp
#include <bits/stdc++.h>
using namespace std;
// CSES 1081 - Common Divisors: maximum gcd(a[i], a[j]) over pairs i < j.
int main() {
    ios::sync_with_stdio(false); cin.tie(nullptr);
    const int M = 1000000;
    int n; cin >> n;
    vector<int> cnt(M + 1, 0);
    for (int i = 0; i < n; i++) { int x; cin >> x; cnt[x]++; }
    for (int g = M; g >= 1; g--) {
        int multiples = 0;
        for (int k = g; k <= M; k += g) multiples += cnt[k];
        if (multiples >= 2) { cout << g << "\n"; return 0; }
    }
}
```

8. Common mistakes

- **Floating-point square roots.** `for (d = 1; d <= sqrt(n); d++)` recomputes `sqrt` every iteration and can be off by one for large n. Use `d * d <= n` with `long long d`.
- **Overflow in `i * i`.** In a sieve with n near $2 \cdot 10^9$, or in trial division with `int`, `i * i` overflows. Cast to `long long`.
- **Forgetting the leftover prime** after trial division (`if (n > 1)`), which loses the largest prime factor of numbers like $2 \cdot 1000000007$.
- **Double-counting √n** when listing divisor pairs of a perfect square.
- **lcm overflow**: `a * b / gcd(a, b)`. Also, the lcm of many numbers grows extremely fast — check whether it can exceed 10^{18} and cap it if the problem allows.
- **Treating 1 as prime**, or forgetting that 0 and 1 need special cases (`isPrime[1]`, `gcd(0, x) = x`).
- **Memory**: `int spf[10'000'001]` is 40 MB, `long long` would be 80 MB. Use the smallest type that fits.
- **Re-sieving per test case.** With many test cases, sieve once up to the maximum value before reading input.

9. Practice problems

Warm-up:

1. **Codeforces 26A — Almost Prime** — https://codeforces.com/problemset/problem/26/A (count numbers with exactly two distinct prime factors).
2. **Codeforces 230B — T-primes** — https://codeforces.com/problemset/problem/230/B (x = p^2; sieve up to 10^6).

3. **Codeforces 1350A — Orac and Factors** — https://codeforces.com/problemset/problem/1350/A (smallest divisor > 1).
4. **CSES 1713 — Counting Divisors** — https://cses.fi/problemset/task/1713

Core:

5. **CSES 1081 — Common Divisors** — https://cses.fi/problemset/task/1081
6. **Codeforces 776B — Sherlock and his girlfriend** — https://codeforces.com/problemset/problem/776/B (primes vs composites → 2 colours).
7. **Codeforces 1458A — Row GCD** — https://codeforces.com/problemset/problem/1458/A (gcd of differences).
8. **AtCoder ABC 177 E — Coprime** — https://atcoder.jp/contests/abc177/tasks/abc177_e (SPF factorization, count prime occurrences).
9. **SPOJ PRIME1 — Prime Generator** — https://www.spoj.com/problems/PRIME1/ (segmented sieve).

Challenge:

10. **Codeforces 1047C — Enlarge GCD** — https://codeforces.com/problemset/problem/1047/C (divide by the gcd, then count prime-factor frequencies with an SPF sieve up to $1.5 \cdot 10^7$).
11. **CSES 1082 — Sum of Divisors** — https://cses.fi/problemset/task/1082 ($\Sigma\ \sigma(i)$ for $i \le 10^{12}$; group equal values of $\lfloor n/i \rfloor$, needs modular arithmetic from Week 11).

10. Further reading

- Antti Laaksonen, *Competitive Programmer's Handbook*, Chapter 21 "Number theory" (primes and factors, sieve, Euclid's algorithm): https://cses.fi/book/book.pdf
- CP-Algorithms: "Sieve of Eratosthenes" https://cp-algorithms.com/algebra/sieve-of-eratosthenes.html, "Linear Sieve" https://cp-algorithms.com/algebra/prime-sieve-linear.html, "Euclidean algorithm" https://cp-algorithms.com/algebra/euclid-algorithm.html, "Integer factorization" https://cp-algorithms.com/algebra/factorization.html, "Number of divisors / sum of divisors" https://cp-algorithms.com/algebra/divisors.html
- USACO Guide, Gold "Divisibility": https://usaco.guide/gold/divisibility
- Next: Week 11 (modular arithmetic, inverses, nCr mod p); Regionals track "Number theory for regionals" (extended Euclid, CRT, Euler φ).